

Research on Formal Specification Generation Methods for Airborne Software Testing

Jiahao Wen
University of Electronic Science and
Technology of China
Chengdu, Sichuan, China
1831202394@qq.com

Liangcai Wu
Aviation Key Laboratory of Science
and Technology on Aerospace Vehicle
Chengdu, Sichuan, China
wulc@avic.com

Xiaopin Chen^{*}
University of Electronic Science and
Technology of China
Chengdu, Sichuan, China
xpchen@uestc.edu.cn

Xianggang Zhang
University of Electronic Science and
Technology of China
Chengdu, Sichuan, China
csxgzhang@uestc.edu.cn

Xiaobo Ma
Xi'an Aeronautical Computing
Technology Research Institute of
AVIC
Xi'an, Shaanxi, China
11873275@qq.com

Abstract

This experimental study investigates the feasibility and effectiveness of automatically translating natural language requirements into ANSI/ISO C Specification Language (ACSL) for formal verification of airborne software using Large Language Models (LLM) combined with Retrieval-Enhanced Generation (RAG) technology. The proposed workflow constructs a loop of model generation, model verification, and model refinement in ACSL. Experimental results show that, compared to manually writing specifications, the LLM+RAG workflow significantly shortens specification generation time, substantially reduces the high error rate caused by using LLM alone, and maintains higher syntactic and functional intent correctness. The study found that the failure modes of the generated specifications mainly involve logical inconsistencies rather than syntactic errors, and the model's self-censorship mechanism effectively mitigates these logical inconsistency problems. The results also indicate that this workflow is most effective for requirements with clear patterns and precedents in the knowledge base, providing a promising automated method for the verification of airborne safety-critical systems.

CCS Concepts

• **Computing methodologies** → Artificial intelligenceNatural; language processing; Machine translation.

Keywords

Formal Specification, LLM, RAG, ACSL

ACM Reference Format:

Jiahao Wen, Liangcai Wu, Xiaopin Chen, Xianggang Zhang, and Xiaobo Ma. 2026. Research on Formal Specification Generation Methods for Airborne Software Testing. In *2026 6th International Conference on*

^{*}Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CNSSE 2026, Qingdao, China*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2195-3/2026/02

<https://doi.org/10.1145/3803633.3803662>

Computer Network Security and Software Engineering (CNSSE 2026), February 06–08, 2026, Qingdao, China. ACM, New York, NY, USA, 6 pages.
<https://doi.org/10.1145/3803633.3803662>

1 Introduction

With the increasing demands for reliability and verification efficiency in airborne software testing, formal methods have become an important means of ensuring its correctness. However, the traditional method of relying on manual writing of formal specifications is time-consuming, labor-intensive, and prone to errors, becoming a major bottleneck for large-scale applications. Meanwhile, the rapid development of large language models (LLMs) has demonstrated their great potential in code generation and program understanding, such as the emergence of GPT4 [1]. Cao et al. have demonstrated that LLMs can assist in generating proof steps and formal structures [2]. However, these general-purpose models often struggle with the strict syntax of specific languages such as ACSL and lack in-depth semantic understanding of avionics logic, often leading to "illusions" or unverifiable specifications [3].

To address the large model illusion and in-depth domain semantic understanding, Retrieval-Enhanced Generation (RAG) has been introduced as a key solution combining LLMs with factual domain knowledge [4]. Studies by Yang et al. [5] and Guo et al. [6] have demonstrated that RAGs can significantly improve the accuracy and contextual relevance of generated content in software engineering tasks by retrieving relevant code snippets, API documents, or specification templates. However, the code logic of the RAG library itself may deviate significantly from actual software requirements, thus requiring feedback modifications to the library itself [7]. In other words, there is currently a lack of an effective closed-loop mechanism to iteratively correct logical inconsistencies [9].

To address these shortcomings, this paper proposes a closed-loop framework that integrates retrieval-enhanced generation and automated formal verification. First, natural language requirements are structurally decomposed through prompt word engineering; second, relevant ACSL patterns and historical specifications are retrieved from the constructed aerospace domain knowledge base using RAG to enhance the accuracy of the generated context; finally,

formal tools are used for verification, and the modified specifications are added to or modified in the RAG library to improve the overall reliability and verification efficiency of the framework. The main contributions of this paper are as follows:

- We systematically analyze the limitations of current specification generation methods, pointing out the efficiency bottleneck of purely manual methods, the high illusion rate of purely LLM methods, and the core defect of existing RAG methods lacking a verification feedback closed loop.
- We propose an automated generation method for formal specifications of airborne software. This method deeply integrates retrieval-enhanced generation with large language models and formal verification toolchains, significantly improving the efficiency and reliability of specification generation.
- We validated our method using real-world datasets, demonstrating its engineering usability and reliability.

The remaining sections of this paper are structured as follows: Part II outlines the research background and problem definition; Part III details the specification generation framework based on LLM+RAG and closed-loop validation; Part IV presents experimental results and analysis; and Part V summarizes the entire paper and proposes future research directions.

2 Formal verification background

Formal verification is a core method for airborne software safety [8]. It uses rigorous mathematical logic to prove that software design or code conforms to its specifications, thus exceeding the coverage limitations of traditional testing.

The ANSI/ISO C Specification Language (ACSL) is a behavioral specification language designed specifically for this purpose. It allows engineers to embed assertions such as preconditions, postconditions, loop invariants, and data invariants directly into C source code as formal comments. ACSL specifications translate potentially ambiguous requirements described in natural language into machine-readable and rigorously sound mathematical propositions, providing precise input for subsequent automated verification.

However, writing correct and complete ACSL specifications is itself a specialized and time-consuming task. To achieve automated verification of the generated ACSL specifications, this study uses Frama-C and its WP plugin as a verification toolchain. Frama-C is a static analysis and formal verification platform based on the C language. Its WP (Weakest Premise) plugin can automatically calculate the weakest premises of a program based on Hall logic and use an SMT (Satisfiability Modular Theory) solver to verify whether the ACSL specification is true in all execution paths of the code. Frama-C was chosen because of its natural integration with the ACSL language, excellent support for industrial-grade C code, and mature and reliable reasoning capabilities, making it one of the de facto standard tools for formal verification of airborne C programs. Therefore, our research ultimately utilizes the verification results of Frama-C to evaluate and improve the generation quality of the ACSL specification.

3 Methodology

To address the ambiguity of natural language and the stringent grammatical requirements of formal verification, this paper proposes a closed-loop framework for generating ACSL specifications. This framework integrates a Large Language Model (LLM), Retrieval Augmentation Generation (RAG), and a Frama-C verification toolchain. As shown in Figure 1, the entire workflow comprises five distinct stages: requirements structuring, RAG-based formal specification generation, formal verification, correction and logging of erroneous data, and fine-tuning the model with correct data.

The process begins with LLM parsing and cue word engineering the requirements to identify key elements. Next, the RAG module retrieves relevant domain knowledge to provide context. The model then generates an initial specification and performs self-checking, correcting obvious grammatical errors. Subsequently, the self-checked specification is validated using Frama-C. Validated specifications are stored directly, while those that fail validation are reviewed and corrected by domain experts. Both results are aggregated into a dataset for fine-tuning the model, improving its generation speed and reliability.

3.1 Requirement Preprocessing and Structural Decomposition

The process initiates with the ingestion of the original airborne software requirement document, denoted as R_{raw} . Since natural language often contains redundant information, a Prompt Engineering module is employed to map the text into a formal intermediate representation. Let $\mathcal{P}$ be the prompt engineering transformation function. The goal is to extract a structured set S , which is defined as:

$$S = P(R_{raw}; \theta_p) \quad (1)$$

where θ_p represents the structural prompt template. The output S is a tuple (V, C_p, C_l) , where V is the set of variables, C_p denotes physical constraints and C_l represents logical assertions.

3.1.1 Backend code. class RequirementDecomposer:

```
def decompose\_natural\_requirement(self, raw\_text: str) -> D
    variables = self.extract\_variables(raw\_text)
    constraints = self.extract\_constraints(raw\_text)
    logic = self.extract\_logic(raw\_text)
    structured\_req = {
        "variables": variables,
        "physical\_constraints": constraints,
        "logical\_assertions": logic}
    return structured\_req
```

The main code is primarily an LLM prompt word template. Its core function is to automatically parse and transform fuzzy natural language requirements into processable, structured keyword data through a set of structured instructions, providing accurate input for subsequent RAG retrieval and reduction generation.

3.2 Domain Knowledge Retrieval via RAG

To reduce the illusion rate and semantic error of large models and to transform large models into ACSL generation experts, we introduce Retrieval Augmented Generation for assistance. The structured requirement S is converted into a query embedding vector,

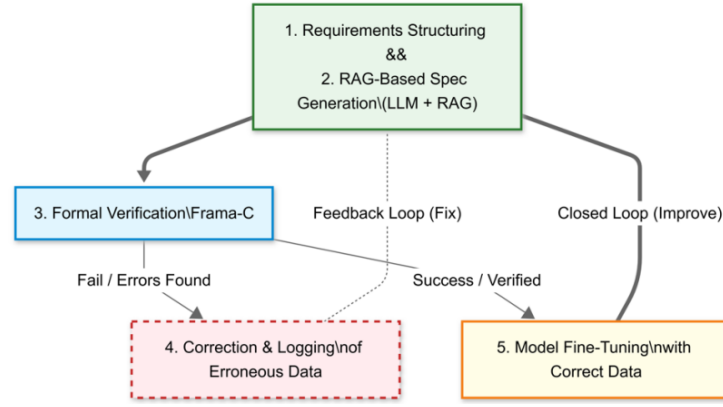


Figure 1: The flowchart of the formal specification generation and verification framework.

denoted as q . The system retrieves the most relevant knowledge chunks K_i from the vector database by calculating the Cosine Similarity. The relevance score ϕ_i is calculated as:

$$\phi_i = \frac{q \cdot K_i}{\|q\| \cdot \|K_i\|} \quad (2)$$

where k_i is the embedding vector of chunk K_i . The final context $\mathcal{C}$ consists of the top- N chunks where the score exceeds a predefined threshold τ :

$$\mathcal{C} = \cup_{i=1}^N \{K_i \mid \phi_i \geq \tau\} \quad (3)$$

where q Implemented using an embedding model, In Vector Database the formula $\frac{q \cdot K_i}{\|q\| \cdot \|K_i\|}$ is the built-in Inner Product or "Cosine" metric, τ is a hyperparameter in the retrieval configuration, Chunks below this score are discarded to prevent noise injection.

3.2.1 Backend code. class KnowledgeRetriever:

```
def retrieve_relevant_context(self, structured_req: Dict) -> List[str]:
    query_vector = self.embed_query(structured_req)
    similarities = [self.cosine_similarity(query_vector, kb_vector)
                    for kb_vector in knowledge_base_vectors]
    relevant_knowledge = self.filter_top_n(similarities,
    threshold=0.7, top_n=5)
    return relevant_knowledge
```

This part of the code implements a RAG-based retrieval mechanism. It first uses a domain-optimized embedding model to transform structured requirements into query vectors. The retrieval process then uses cosine similarity to rank knowledge blocks and selects the top N results.

3.3 Context-Aware Generation and Self-Check

The LLM generates the ACSL specification by maximizing the conditional probability of the output token sequence y , given the structured input S and context $\mathcal{C}$. The optimization goal is:

$$y^* = \operatorname{argmax}_P(y|S, \mathcal{C}; \Theta) \quad (4)$$

where Θ represents the LLM parameters.

To ensure syntactic correctness, a post-generation validation function, $\text{Check}(y)$, is introduced. The final draft specification y_{final} is determined by:

$$y_{\text{final}} = \begin{cases} y^*, & \text{if } \text{Check}(y^*) = \text{True} \\ \text{Refine}(y^*), & \text{otherwise} \end{cases} \quad (5)$$

where $\text{Refine}(\cdot)$ denotes a secondary correction pass. This corresponds to the LLM generation process. We set the Temperature parameter to a low value (e.g., 0.1) to force the model to choose the most probable tokens (greedy decoding), ensuring determinism. $\text{Check}(y^*)$: Implemented as a Python script using Regular Expressions (Regex) or a syntax parser. Refine Loop is a "Conditional Edge". If $\text{Check}()$ returns False, the flow routes back to the LLM with an error prompt; otherwise, it proceeds to output.

3.3.1 Backend code. class ACSLSpecGenerator:

```
def generate_and_verify(self, structured_req: Dict, context: List[str]) -> str:
    draft_spec = self.llm_generate(structured_req, context,
    temperature=0.1)
    if self.syntax_check(draft_spec) and self.logic_consistency_check(draft_spec):
        return draft_spec
    else:
        error_feedback = self.analyze_errors(draft_spec)
        refined_spec = self.refine_with_feedback(draft_spec, error_feedback)
        return refined_spec
    def llm_generate(self, structured_req: Dict, context: List[str],
    temperature: float) -> str:
        prompt = self.build_generation_prompt(structured_req, context)
        specification = self.llm.generate(prompt, temperature=temperature)
        return specification
```

This part of the code primarily receives structured requirements and retrieved contextual knowledge from previous steps, then generates a preliminary ACSL specification draft. Next, it automatically performs syntax and basic logic checks on this draft. If the checks

pass, the draft becomes the final output; if it fails, the system analyzes errors, generates correction suggestions, and automatically regenerates an improved specification.

3.4 Automated Verification with Frama-C

The generated specification is mapped to the C source code. The verification relies on Hoare Logic, where the validity of the triple $\{P\}C\{Q\}$ is checked. The Frama-C WP plugin computes the Weakest Precondition, denoted as $wp(C, Q)$. The verification condition is satisfied if and only if the implication holds:

$$P \Rightarrow wp(C, Q) \quad (6)$$

The verification result, V_{res} , is a boolean value obtained from the Satisfiability Modulo Theories (SMT) solver:

$$V_{res} = \text{SMT_Verify}(P \Rightarrow wp(C, Q)) \quad (7)$$

3.5 Feedback Loop and Model Fine-tuning

The final stage of the framework establishes a closed-loop feedback mechanism to improve model performance. This process relies on accumulating high-quality datasets from actual validation results. The output of the dataset contains two data streams:

- (1) Positive samples: Specifications validated through Frama-C are automatically marked as valid samples.
- (2) Negative samples with corrections: Specifications that fail validation are marked for expert review. Domain experts analyze the reasons for failure and manually correct the ACSL code. Once these corrected specifications pass validation, they become high-quality positive samples.

All datasets are integrated to fine-tune the large language model, enabling it to better understand the syntax of ACSL and the specific functional patterns of the onboard software, thereby reducing the need for manual intervention in future iteration cycles.

4 Experimental

To systematically evaluate the performance of the proposed method, this study designed a comparative experiment comprising three groups of different requirement modules. These groups represent varying levels of logic complexity found in typical avionics software. The experiment compares three specification generation approaches: Manual Expert Writing (M1), LLM without RAG (M2), and the proposed LLM + RAG Framework (M3).

Figure 2 illustrates a concrete example of how the LLM+RAG framework handles natural language requirements. A user query for "Write ACSL specifications for the aircraft brake status management module code" first triggers the RAG module, which retrieves the most relevant domain knowledge (similarity score: 0.63) from a local knowledge base that provides basic ACSL syntax templates (e.g., usage of 'requires' and 'ensures'). Guided by the retrieved context, LLM generates a structured and verifiable ACSL specification. The output includes a complete functional contract: preconditions and postconditions, as well as action clauses for different patterns. This example clearly demonstrates the convenience of the LLM+RAG framework, significantly reducing the time spent manually writing specifications.

```

Question: Write an ACSL specification for an aircraft brake status management module code.

Searching local knowledge base...

Found 1 relevant knowledge point.

Knowledge point 1 (similarity: 0.63):
ACSL specification is used for formal verification of C programs. Preconditions use the requires keyword, postconditions use the ensures keyword...

Generated ACSL specification:
=====
Calling the large model to generate the specification.
The following is an example of the generated ACSL specification based on the requirements:

// Preconditions
requires \valid(brake_status) && 0 <= brake_pressure && brake_pressure <=
MAX_BRAKE_PRESSURE;
// The brake status pointer must be valid, and the brake pressure must be within the legal range

ensures \result == BRAKE_STATE_NORMAL || \result == BRAKE_STATE_OVERHEATED || \result
BRAKE_STATE_FAILURE;
// The function return value must be one of normal, overheated, or failure states

assigns *brake_status;
// This function only modifies the value pointed to by the brake status pointer

```

Figure 2: Example of LLM + RAG processing

4.1 Dataset classification

To ensure a rigorous evaluation, the dataset is categorized into three levels based on Cyclomatic Complexity (CC), a quantitative measure of logical intricacy:

- (1) Group A (Simple, $CC \leq 5$): Linear utility functions with minimal branching.
- (2) Group B (Medium, $6 \leq CC \leq 15$): Logic involving multiple conditional branches and state updates.
- (3) Group C (Complex, $CC > 15$): Algorithmic modules involving nested loops, arrays, and non-linear computations.

4.2 Evaluation Metrics

The performance is assessed using three key engineering metrics:

- Generation Time (T_{gen}): The average time required to produce the initial specification draft.

- Hallucination Rate ($R_{hallucination}$): The percentage of generated specifications containing "hallucinated" elements, specifically, non-existent variables, undefined functions, or fabricated ACSL predicates not supported by the Frama-C kernel [10].

- Average Iterations (N_{iter}): The average number of "Generation-Verification-Correction" cycles required to achieve a specification that fully passes Frama-C WP verification. A value of 1.0 indicates a perfect first-pass success.

4.3 Results

This experiment obtained over 90 flight control airborne software module requirements and corresponding code from the Aerospace Key Laboratory. Test cases were divided into three groups—simple, medium, and complex—based on the cyclomatic complexity of the requirements. For each group, three methods were used to generate ACSL specifications: M1 (manually written by experts), M2 (pure LLM generation), and M3 (LLM+RAG generation). A timer recorded the time taken from start to first-version specification

Table 1: Performance Comparison across Complexity Levels

Complexity Group	Method	T_gen	R_hallucination	N_iter
Group B	M1 (Manual)	24 min	~0%	1.2
	M2 (LLM)	0.2 min	15%	2.8
	M3 (LLM+RAG)	0.3 min	2%	1.3
	M1	56 min	~0%	1.5
	M2	0.4 min	45%	6.4
	M3	0.6 min	8%	2.0
Group C	M1	183 min	~0%	2.5
	M2	0.8 min	83%	8
	M3	1.2 min	14%	3.5

Table 2: Comparison of the experimental method in this study with research methods in recent years

Experimental	Method	T_gen	R_hallucination	N_iter
Ours	M3(LLM+RAG)	0.5 min	~6%	1.2
Cao et al	M2(LLM)	0.4 min	~58%	2.6

production. Subsequently, all generated specifications were formally verified using the formal verification tool Frama-C and its WP plugin, combined with manual review to statistically analyze the proportion of specifications containing fictitious variables, predicates, or logic as the hallucination rate (R_hallucination). Finally, the average number of "generation-verification-revision" iterations (N_iter) required to obtain a specification that fully passes Frama-C verification was recorded. All tests were conducted in the same environment, under the same version and verification standards, and the final data are statistical results from multiple rounds of experiments.

The ablation experimental results are summarized in Table 1.

To avoid the influence of domain-specific data and highlight the advancements of our proposed method in the field of formal reduction generation, we referenced the ACSL generation method by Cao et al. [2] and conducted comparative experiments using their open-source dataset. The results of the comparative experiments are shown in Table 2.

5 Conclusion

Based on the experimental results and discussions presented above, the conclusions are obtained as below:

(1) The LLM+RAG pipeline significantly enhances engineering efficiency. The experiments demonstrate that for complex functional modules, the generation time is reduced from hours (manual) to minutes, while maintaining a logical consistency score comparable to human experts.

(2) The introduction of domain knowledge via RAG is critical for suppressing model hallucinations. Experimental data shows that for high-complexity modules, the RAG mechanism reduces the hallucination rate from 83% to 14%, ensuring that generated loop invariants and predicates strictly adhere to avionics standards.

(3) The "Generation-Verification-Correction" feedback loop proved effective in ensuring specification validity. The framework

demonstrates stable convergence with an average of 1.3 to 3.5 iterations, significantly reducing the cost of rework and providing a practical pathway for the large-scale application of formal methods in safety-critical systems.

This experimental study demonstrates that RAG's assistance truly makes LLM an expert in the field of automated formal specification generation for airborne software. The combination of LLM, RAG, and formal tools provides a powerful and practical framework and methodology for airborne software specification generation. It successfully addresses key challenges in automation, accuracy, and reliability, offering a promising approach to improving the rigor and efficiency of safety-critical software verification. A drawback is that some time and effort are still required in the initial manual review stage; however, with iterative improvements to the model, the entire model will become more accurate, reducing review time and iteration count.

Acknowledgments

This work was financially supported by the Open Funding of Aviation Key Laboratory of Science and Technology on Aerospace Vehicle and AVIC United Technology Center of Intelligent Decision-making and Coordinated Control Mechanism Model Research.

References

- [1] Bubeck, Sébastien, *et al.* "Sparks of artificial general intelligence: Early experiments with gpt-4." arXiv preprint arXiv:2303.12712 (2023). <https://doi.org/10.48550/arXiv.2303.12712>
- [2] Cao, Jialun, *et al.* "From Informal to Formal-Incorporating and Evaluating LLMs on Natural Language Requirements to Verifiable Formal Proofs." arXiv preprint arXiv:2501.16207 (2025). <https://doi.org/10.48550/arXiv.2501.16207>
- [3] Granberry, George, Wolfgang Ahrendt, and Moa Johansson. "Specify what? Enhancing neural specification synthesis by symbolic methods." International Conference on Integrated Formal Methods. Cham: Springer Nature Switzerland, 2024. <https://doi.org/10.48550/arXiv.2406.15540>
- [4] Lewis, Patrick, *et al.* "Retrieval-augmented generation for knowledge-intensive nlp tasks." Advances in neural information processing systems 33 (2020): 9459-9474. <https://doi.org/10.48550/arXiv.2005.11401>
- [5] Yang, Rui, *et al.* "RAGVA: Engineering retrieval augmented generation-based virtual assistants in practice." arXiv preprint arXiv:2502.14930 (2025). <https://doi.org/10.48550/arXiv.2502.14930>

- org/10.48550/arXiv.2502.14930
- [6] Gu, Wenchao, *et al.* "What to Retrieve for Effective Retrieval-Augmented Code Generation? An Empirical Study and Beyond." arXiv preprint arXiv:2503.20589 (2025). <https://doi.org/10.48550/arXiv.2503.20589>
 - [7] Shi, Freda, *et al.* "Large language models can be easily distracted by irrelevant context." International Conference on Machine Learning. PMLR, 2023. <https://doi.org/10.48550/arXiv.2302.00093>
 - [8] Hoare, Charles Antony Richard. "An axiomatic basis for computer programming." *Communications of the ACM* 12.10 (1969): 576-580. <https://doi.org/10.1145/363235.363259>
 - [9] Lahiri, Shuvendu K., *et al.* "Interactive code generation via test-driven user-intent formalization." arXiv preprint arXiv:2208.05950 (2022). <https://doi.org/10.1145/3656408.3662833>
 - [10] Chen, Mark. "Evaluating large language models trained on code." arXiv preprint arXiv:2107.03374 (2021). <https://doi.org/10.48550/arXiv.2107.03374>